

OE-EC803A

Internet of Things

Combined Module Notes

Modules 1 – 6

Semester VIII • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

Disclaimer / Study Guide Notice:

These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Biraj Sarkar

IN ASSOCIATION WITH

MAKAUT Wingman • MAKAUT Future Minds

Cooch Behar Government Engineering College

Exam-Ready Notes

Contents

Module 1: The Internet of Things: An Overview	4
1 The Internet of Things: An Overview	4
1.1 The Flavour of the Internet of Things	4
1.2 The Technology of the Internet of Things	4
1.3 Enchanted Objects	5
1.4 Who is Making the Internet of Things?	6
2 Design Principles for Connected Devices	6
2.1 Calm and Ambient Technology	6
2.2 Magic as Metaphor	6
2.3 Web Thinking for Connected Devices	7
2.4 Affordances	7
Module 2: Internet Communications Overview	10
1 Internet Communications Overview	10
1.1 Transport Layer: TCP vs. UDP	10
2 IP Addressing and Routing	11
2.1 IP Addresses and DNS	11
2.2 IPv4 vs. IPv6 in IoT	11
2.3 MAC Addresses	11
2.4 Ports	12
3 Application Layer Protocols in IoT	12
3.1 MQTT (Message Queuing Telemetry Transport)	12
3.2 CoAP (Constrained Application Protocol)	12
Module 3: Thinking About Prototyping	17
1 Thinking About Prototyping	17
1.1 Prototyping Considerations	17
2 Basic Electronics for IoT	17
2.1 Basic Quantities	17
2.2 Signals and Interface Elements	18
3 Prototyping Embedded Platforms	18
3.1 Embedded Computing Basics	18
3.2 Hardware Platforms Overview	19
Module 4: The Physical Prototyping Process	24
1 The Physical Prototyping Process	24
1.1 Prototyping Workflow Steps	24
2 Prototyping Methods	25

2.1	Non-Digital Prototyping (Low-Fidelity)	25
2.2	Laser Cutting (2D Vector Fabrication)	25
2.3	3D Printing (Additive Manufacturing)	25
2.4	CNC Milling (Subtractive Manufacturing)	26
2.5	Repurposing and Recycling	26

Module 5: Prototyping Online Components (APIs) 29

1	Prototyping Online Components (APIs)	29
1.1	RESTful APIs	29
1.2	Writing a New API (Backend Endpoint)	30
1.3	Real-Time Reactions: Polling vs. WebSockets	30
1.4	Other Messaging Protocols	31
2	Techniques for Writing Embedded Code	31
2.1	Memory Management & Heap Fragmentation	31
2.2	Power Management and Battery Life Optimization	31
2.3	Embedded Debugging Techniques	33

Module 6: IoT Business Models & Startups 37

1	IoT Business Models & Startups	37
1.1	A Short History of Business Models	37
1.2	Who is the Business Model For?	37
1.3	Business Model Canvas in IoT	37
1.4	Lean Startup Methodology	38
1.5	Funding Options	38
2	Moving to Manufacture	38
2.1	What are you Producing? & Designing Kits	38
2.2	Designing and Manufacturing PCBs	39
2.3	Mass-Producing Enclosures (Injection Molding)	40
2.4	Hardware Certification	40
2.5	Manufacturing Cost Concepts	41
2.6	Software Scaling and OTA Updates	41
3	Ethics, Security & Environmental Solutions	41
3.1	Characterizing the Internet of Things (Ethical Perspective)	41
3.2	Privacy and Control	41
3.3	Environmental Impact (E-Waste)	42

Module 1

The Internet of Things: An Overview

1. The Internet of Things: An Overview

Internet of Things (IoT)

The Internet of Things (IoT) is a global network of physically interconnected objects (or “things”) embedded with sensors, software, actuators, and network connectivity, enabling them to collect, exchange, and act on data without requiring direct human-to-human or human-to-computer interaction.

1.1 The Flavour of the Internet of Things

IoT bridges the gap between the physical and digital worlds by introducing **digital identity** and **intelligence** to everyday objects.

- **Physical-Digital Convergence:** Traditional devices (like a toaster or street light) are passive. IoT transforms them into active entities that communicate status and receive remote commands.
- **M2M (Machine-to-Machine) vs. IoT:** While M2M refers to point-to-point communication between hardware devices over dedicated lines (often isolated), IoT represents IP-based communication connecting billions of heterogeneous devices globally through standard internet protocols.

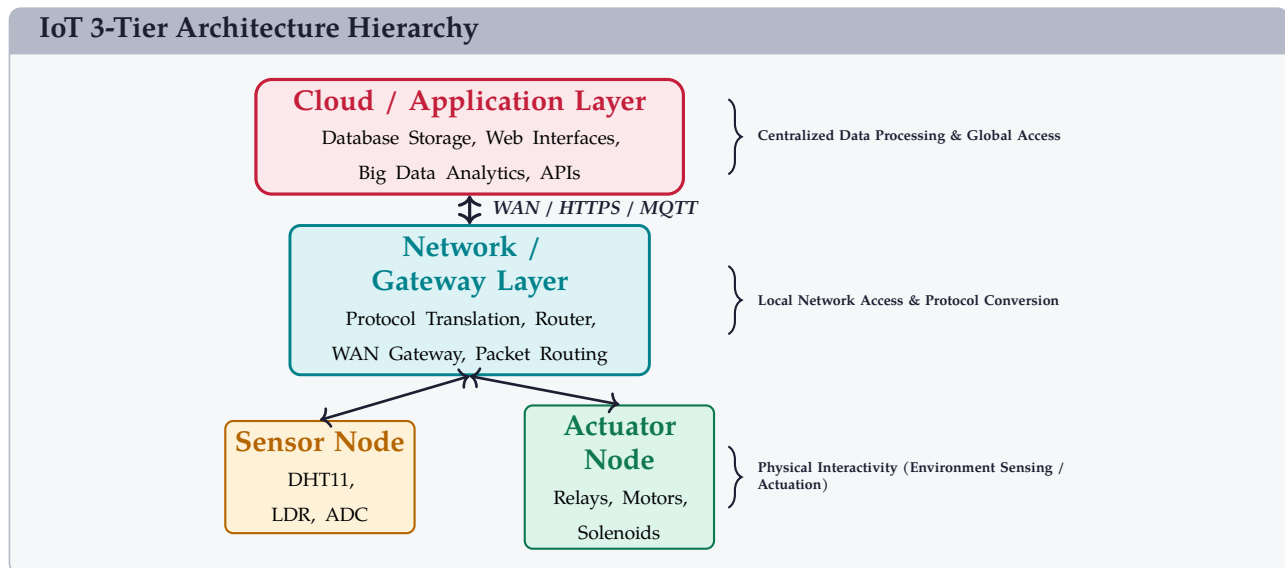
Parameter	Traditional Embedded System	Internet of Things (IoT)
Connectivity	None or local point-to-point (isolated).	IP-based internet connection (global routing).
Data Processing	Local execution on microcontroller.	Distributed processing (Edge + Cloud computing).
Human Interaction	Direct interface (buttons, LCD screen).	Ambient, screenless, or remote app interaction.
Scale	Standalone, singular execution.	Interconnected, heterogeneous device fleets.

1.2 The Technology of the Internet of Things

The IoT system is built upon a multi-tier architectural stack:

1. **Edge Layer (Physical/Perception):** Comprises sensors (LDR, DHT11) to collect parameters and actuators (relays, motors) to execute physical actions.
2. **Network/Gateway Layer:** Connects edge nodes to the cloud. Handles protocol translation, packet routing, and local data filtering.
3. **Cloud/Application Layer:** Analyzes incoming data streams, executes business logic, manages databases, and provides API endpoints and user dashboards.

IoT 3-Tier Architecture Hierarchy



1.3 Enchanted Objects

The term **Enchanted Objects**, coined by MIT media lab researcher David Rose, refers to ordinary physical objects that are enhanced with localized, ambient technology to display information or react to user actions without demanding their full cognitive attention.

- Instead of forcing users to look at a glowing smartphone screen, information is embedded naturally into the physical environment.
- **Example:** An *Ambient Orb* that glows red if the stock market goes down, or blue if it goes up. The information is consumed at a glance, blending into the background.

David Rose classifies these objects according to **six fundamental human desires**:

1. **Omniscience (All-Knowing):** Objects that provide information, such as the *Ambient Umbrella* that glows when it is going to rain.
2. **Telepathy (Communication):** Objects that allow remote emotional connection, like *connected lamps* that light up when touched by a loved one.
3. **Safekeeping (Protection):** Objects that track security, like smart door locks or GPS-connected wallets.
4. **Health and Vitality:** Wearables or pill bottles that track dosage compliance and body vitals.
5. **Teleportation (Locomotion):** Devices that bridge spatial distance, such as smart cameras or remote-controlled vehicles.
6. **Expression (Creativity):** Instruments and smart surfaces that adapt dynamically to user expressions.

1.4 Who is Making the Internet of Things?

The IoT ecosystem is built by a diverse group of creators across three main levels:

- **Makers and Hobbyists:** Individuals using open-source hardware (Arduino, Raspberry Pi) and software libraries to build custom DIY projects. They drive rapid community innovation.
- **Startups:** Lean companies developing niche connected products (e.g. smart pet feeders, fitness wearables) and leveraging crowdfunding to prove market demand.
- **Tech Giants and Enterprises:** Large corporations (like Bosch, Cisco, Siemens, Google, Apple, and AWS) that design industrial IoT (IIoT) platforms, establish global protocols, and build massive cloud infrastructure to manage billions of nodes.

2. Design Principles for Connected Devices

Designing connected products requires moving away from the mouse-and-keyboard paradigm toward natural physical interfaces.

2.1 Calm and Ambient Technology

Calm Technology

Calm Technology, formulated by Mark Weiser and John Seely Brown, is hardware/software design that informs the user but remains in the periphery of their attention, moving smoothly between the center and the margin of user focus.

- **Periphery Concept:** The human brain processes peripheral cues constantly (like the hum of an engine or wind blowing). Calm design uses these peripheral channels to present digital information, minimizing cognitive load.
- **Ambient Displays:** Use light, sound, or physical motion to convey information subtly (e.g., a smart umbrella handle that glows when rain is in the local forecast).

Mark Weiser and John Seely Brown characterized the shift in computing history across three main eras:

- **Mainframe Era:** One computer shared by many people. High complexity and high cognitive distance.
- **Personal Computer (PC) Era:** One computer per person. Active screen attention, leading to information overload.
- **Ubiquitous/Calm Computing Era:** Many computers per person. Technology recedes into the physical background, operating in our peripheral attention.

2.2 Magic as Metaphor

When designing connected objects, users often conceptualize digital capabilities as “magic” (e.g. a mirror that displays weather, or a key that tracks its location).

- Metaphors from fairy tales (such as crystal balls, magic wands, and mirrors) serve as design patterns for natural user interfaces.
- The technology should remain invisible; the user interacts directly with the physical object, not the screen/interface.

2.3 Web Thinking for Connected Devices

Applying Web principles to physical hardware is called the **Web of Things (WoT)**:

- **Loose Coupling:** Connected devices should be modular and independent. Changing the cloud backend should not brick the physical device.
- **RESTful Architectures:** Representing physical actions as resources using standard HTTP verbs (e.g., 'POST /livingroom/light' to turn on a lamp).
- **Open Standards:** Prefer open APIs, JSON, and TCP/IP protocols over proprietary closed stacks to prevent vendor lock-in.

2.4 Affordances

Affordance

An **Affordance** is the physical property of an object that suggests how it should be used or interacted with (e.g., a handle affords pulling, and a button affords pushing).

In IoT design, physical affordances must match digital functions. If a smart lamp requires twisting to dim, the physical knob must feel like a dimming control, and the digital latency must be low enough to maintain the illusion of physical causality.

Quick Revision — Module 1

Introduction & Design Principles | OE-EC803A

One-Line Definitions

- **IoT:** Interconnected network of physical objects embedded with sensors and connectivity.
- **Enchanted Object:** A physical item enhanced with subtle digital properties to make it feel “magical.”
- **Calm Technology:** Technology designed to sit in the user’s peripheral attention until needed.
- **Affordance:** Physical shape or properties of an object suggesting its interface usage.
- **Web of Things (WoT):** Extending Web standards (HTTP, REST, JSON) to integrate IoT devices.

Frequently Asked / Viva Questions

Q1. What is the difference between IoT and M2M?

M2M (Machine-to-Machine) is usually point-to-point, closed-network hardware communication over dedicated links (e.g., cellular/radio), while IoT is a globally open network of heterogeneous devices interacting using standardized IP protocols.

Q2. Explain David Rose’s concept of “Enchanted Objects.”

David Rose describes enchanted objects as ordinary physical items (like mirrors, umbrellas, or pens) embedded with simple, glanceable digital capabilities. Instead of screens commanding our focus, these items use ambient cues to blend information into our physical surroundings.

Q3. How does Calm Technology process information?

Calm technology relies on the human brain’s ability to process peripheral cues. It displays digital status parameters in the margins of attention (like an ambient light color change). If the parameter changes critically, it draws the user’s focus to the center.

Q4. What is an “affordance” in physical device design?

An affordance is a visual or physical cue indicating how an object operates. For example, a round dial suggests rotation. In IoT design, physical controls must afford their digital actions to make the interface intuitive.

Q5. What does the phrase “Magic as Metaphor” mean in IoT design?

It means designing connected objects that behave like mythical items (e.g., a compass pointing to a friend, or a key that remembers where it is). It simplifies the user’s mental model by mapping complex technology to familiar magical concepts.

Frequently Asked / Viva Questions (contd.)

Q6. Why is open API standard design critical in IoT?

Proprietary closed APIs isolate devices and lead to vendor lock-in. If the vendor shuts down their cloud server, the device is bricked. Open APIs and standards allow interoperability and support local control independent of specific cloud stacks.

Q7. What is the Web of Things (WoT)?

The Web of Things is a framework that uses standard web application layer protocols (HTTP, REST, WebSockets, JSON) to interface and control IoT edge hardware, making them as easy to access as regular web documents.

Q8. What are the typical layers of an IoT system?

(1) Perception/Edge Layer (collects data via sensors), (2) Network/Gateway Layer (handles routing and protocol translation), and (3) Application/Cloud Layer (analyzes data and executes business logic).

Comparison Snapshot

Calm Technology vs. Screen-Based Technology:

Feature	Calm Technology	Screen-Based Technology
Attention Demand	Relies on peripheral attention.	Demands full visual and cognitive attention.
Context Integration	Integrates seamlessly into physical objects.	Requires dedicated, isolated visual displays.
Cognitive Load	Low; processes cues ambiently.	High; causes screen fatigue and information overload.
Example	Smart umbrella glowing when rain is near.	Phone notification showing rain forecast percentage.

Module 2

Internet Communications Overview

1. Internet Communications Overview

IoT devices rely on internet communication stacks to transmit sensor data and receive commands. The dominant model is the **TCP/IP Protocol Suite**.

TCP/IP Protocol Suite

The **TCP/IP Protocol Suite** is a four-layer conceptual model for network communication. It consists of the Link Layer, Internet Layer (IP), Transport Layer (TCP/UDP), and Application Layer (HTTP/CoAP/MQTT).

1.1 Transport Layer: TCP vs. UDP

IoT edge devices choose transport protocols based on power, latency, and reliability requirements:

- **TCP (Transmission Control Protocol):** A connection-oriented, reliable protocol. It guarantees in-order delivery of data using handshakes, acknowledgments, and retransmission mechanisms. The drawback is high header overhead (20 bytes) and higher power consumption, which is challenging for constrained devices.
- **UDP (User Datagram Protocol):** A connectionless, lightweight transport protocol. It sends packets (datagrams) without verifying if the receiver is ready or if the packets arrive. It has minimal header overhead (8 bytes) and low latency, making it ideal for battery-powered sensor nodes (often combined with CoAP).

TCP Three-Way Handshake Connection Lifecycle

Establishing a reliable TCP connection requires a three-way handshake before any application data is sent:

1. **SYN (Synchronize):** The client sends a segment with the SYN flag set and an initial sequence number ($Seq = x$) to request a connection.
2. **SYN-ACK (Synchronize-Acknowledge):** The server replies with a segment where both SYN and ACK flags are set, acknowledging the client's sequence ($Ack = x + 1$) and sending its own sequence number ($Seq = y$).
3. **ACK (Acknowledge):** The client sends a final segment acknowledging the server's sequence ($Ack = y + 1$). The connection is now **ESTABLISHED**.

For teardown, a four-step handshake using **FIN** (Finish) and **ACK** segments is executed to close both directions of the full-duplex link.

2. IP Addressing and Routing

Every node connected to the Internet needs a unique address for routing packets.

2.1 IP Addresses and DNS

- **DNS (Domain Name System):** Translates human-readable domain names (e.g. 'api.weather.com') into machine-readable IP addresses.
- **Static IP Assignment:** The address is manually configured on the device. Ideal for servers and local gateways but hard to scale.
- **Dynamic IP Assignment (DHCP):** A DHCP server automatically assigns a temporary IP address to devices when they connect to the network. Essential for scaling large fleets of consumer IoT devices.

2.2 IPv4 vs. IPv6 in IoT

IPv6 Importance in IoT

IPv6 (Internet Protocol version 6) uses 128-bit addresses, yielding 3.4×10^{38} unique addresses, compared to the 32-bit (4.3 billion) address pool of **IPv4**. IPv6 is critical for IoT because it allows every smart object on Earth to have a unique, globally routable public IP address, eliminating the need for complex Network Address Translation (NAT) gateways.

Feature	IPv4 Header	IPv6 Header
Header Size	Variable (20 to 60 bytes).	Fixed (40 bytes).
Address Size	32 bits (4 bytes).	128 bits (16 bytes).
Fields Count	12 fields.	Only 8 fields.
Checksum Field	Present (must be recomputed at each hop).	Removed (handled by Layer 2 and Layer 4).
Fragmentation	Handled by routers and sending hosts.	Handled only by sending hosts (PMTU discovery).

2.3 MAC Addresses

A **MAC (Media Access Control) Address** is a unique, 48-bit physical hardware identifier burned into a network interface card (NIC) by the manufacturer. While IP addresses change based on location/network, the MAC address remains constant, uniquely identifying the hardware locally.

2.4 Ports

Ports are 16-bit logical identifiers at the Transport Layer used to route data packets to specific software applications running on a host. Common IoT port assignments include HTTP (80), MQTT (1883), CoAP (5683), and HTTPS/MQTTs (443/8883).

3. Application Layer Protocols in IoT

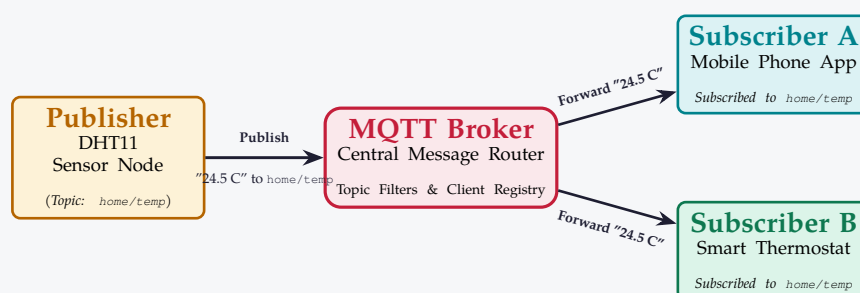
To handle resource constraints (memory, bandwidth, power), specialized application layer protocols are used instead of heavy web HTTP.

3.1 MQTT (Message Queuing Telemetry Transport)

MQTT is a lightweight, broker-based publish-subscribe messaging protocol designed for high-latency, low-bandwidth networks.

- **Clients:** Publish messages to topics or subscribe to topics to receive messages.
- **Broker:** The central server that receives published messages and distributes them to all subscribed clients.
- **QoS (Quality of Service) Levels:**
 - *QoS 0 (At most once):* Fire-and-forget; no handshake, no guaranteed delivery. Minimal overhead.
 - *QoS 1 (At least once):* Guaranteed delivery but duplicates may arrive.
 - Client sends a `PUBLISH` message containing a unique Packet Identifier.
 - Broker stores and forwards the message, replying with a `PUBACK` acknowledgment. If the client doesn't receive `PUBACK` before a timeout, it re-transmits.
 - *QoS 2 (Exactly once):* Four-step handshake guarantees delivery with zero duplicates.
 - Client sends `PUBLISH`.
 - Broker stores packet ID and sends `PUBREC` (Publish Received).
 - Client receives `PUBREC` and sends `PUBREL` (Publish Release) to unlock broker storage.
 - Broker processes/distributes message, replying with `PUBCOMP` (Publish Complete).

MQTT Publish-Subscribe Architecture

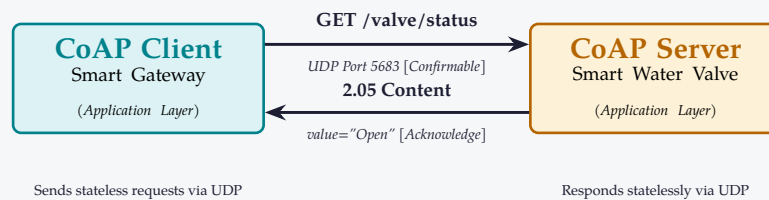


3.2 CoAP (Constrained Application Protocol)

CoAP is a web transfer protocol designed for constrained nodes and networks:

- It uses a RESTful request-response model mapping to standard HTTP methods (GET, POST, PUT, DELETE).
- Unlike HTTP, CoAP runs over **UDP** instead of TCP, dramatically reducing connection handshakes and packet overhead.

CoAP Request-Response over UDP



Quick Revision — Module 2

Networking & Protocols | OE-EC803A

Key Protocols and Specs at a Glance

IPv4 Address Length	32 bits (4 bytes) → e.g., 192.168.1.10
IPv6 Address Length	128 bits (16 bytes) → e.g., 2001 : 0db8 : 85a3 :: 8a2e : 0370 : 7334
MQTT Ports	Port 1883 (TCP, unencrypted) and Port 8883 (Encrypted SSL/TLS)
CoAP Port	Port 5683 (UDP, unencrypted) and Port 5684 (DTLS security)

One-Line Definitions

- **TCP/IP Model:** A standardized suite of communication protocols used to connect devices on the internet.
- **QoS (MQTT):** Quality of Service agreement determining the reliability and handshake level of message delivery.
- **MAC Address:** A permanent, unique physical address burned into the hardware interface card.
- **CoAP:** A lightweight RESTful transfer protocol running over UDP for constrained networks.
- **DHCP:** An automated protocol that dynamically assigns IP configurations to joining devices.

Frequently Asked / Viva Questions

Q1. Why is TCP preferred for applications like web browsing, but UDP is preferred for real-time sensor streams?

TCP is connection-oriented and guarantees that every packet is received in order through acknowledgments. This prevents page layout distortion but adds latency. UDP sends packets without handshakes. It is faster, has less overhead, and is ideal for sensor streams where losing an occasional packet is acceptable but latency must be low.

Q2. Why is IPv6 essential for the growth of IoT?

IPv4 only supports 4.3 billion addresses, which have already been exhausted. IoT connects tens of billions of smart objects. IPv6 provides 128-bit addresses (3.4×10^{38}), allowing every individual device to have a globally unique IP address without routing behind NAT gateways.

Frequently Asked / Viva Questions (contd.)**Q3. How does the publish-subscribe model in MQTT differ from the client-server model in HTTP?**

In HTTP (client-server), clients pull data by initiating requests, requiring both devices to be online and active at the same time (tight temporal coupling). In MQTT (publish-subscribe), clients publish data to a broker, which pushes it to subscribers. The publisher and subscriber do not know each other and are decoupled.

Q4. Explain the three QoS levels in MQTT.

(1) QoS 0 (At most once): Sends the message once with zero handshakes. Best for high-frequency, non-critical data. (2) QoS 1 (At least once): Delivers the message but might send duplicates. Requires a puback packet. (3) QoS 2 (Exactly once): Guarantees the message is received exactly once using a 4-packet exchange.

Q5. What is CoAP, and what transport layer protocol does it use?

CoAP (Constrained Application Protocol) is a lightweight web transfer protocol designed for resource-constrained IoT nodes. It mirrors the RESTful model of HTTP (GET/POST) but runs over UDP, reducing header overhead and handshake latency.

Q6. Why do we need a DNS server?

Humans read names (like 'io.adafruit.com'), but routing hardware only reads binary IP addresses. The DNS server acts as a directory that looks up the domain name and returns the active IP address.

Q7. Explain the difference between static and dynamic IP assignment.

Static IP addresses are configured manually and remain fixed, which is useful for local gateways. Dynamic IP addresses are leased temporarily from a pool by a DHCP server, which simplifies administration for large scale deployments.

Q8. What is the difference between a MAC address and an IP address?

A MAC address is a physical, 48-bit identifier assigned permanently to the device hardware by the manufacturer. An IP address is a logical address assigned dynamically by the network to route packets across subnets.

Comparison Snapshot**MQTT vs. CoAP vs. HTTP:**

Feature	MQTT	CoAP	HTTP
Architecture	Publish / Subscribe	Request / Response (REST)	Request / Response (REST)
Transport Layer	TCP	UDP	TCP
Header Size	Minimal (2 bytes)	Small (4 bytes)	Large (Hundreds of bytes)
Security	SSL / TLS	DTLS	SSL / TLS (HTTPS)
Use Case	Continuous telemetry streams.	Low-power, event-driven queries.	Heavy dashboard interfaces.

Module 3

Thinking About Prototyping

1. Thinking About Prototyping

Prototyping is the iterative process of quickly assembling a working model of a device to test assumptions and functionality before entering mass manufacturing.

1.1 Prototyping Considerations

- **Sketching:** Fast, non-functional drawings to visualize hardware forms and user flows.
- **Familiarity:** Choosing tools the designer already knows (e.g. Arduino IDE) to reduce development time, even if other tools are mathematically more optimal.
- **Costs vs. Ease:** Fast prototyping tools (like jumper wires and breadboards) are cheap and easy, but produce bulky, fragile systems. Production-ready designs require custom PCBs.
- **Open Source vs. Closed Source:** Open hardware/software allows developers to adapt existing code libraries and hardware schematics, tapping into massive global communities.

2. Basic Electronics for IoT

IoT edge nodes interface directly with the physical world using sensors and actuators.

2.1 Basic Quantities

Ohm's Law and Electrical Power

Ohm's Law relates voltage (V), current (I), and resistance (R):

$$V = I \cdot R \quad (1)$$

Electrical power (P) in watts is given by:

$$P = V \cdot I = I^2 \cdot R \quad (2)$$

Voltage Dividers and Analog-to-Digital Conversion (ADC)

Voltage Divider Formula: Reduces a high analog voltage (V_{in}) to a safe range (V_{out}) for microcontroller pins:

$$V_{out} = V_{in} \cdot \left(\frac{R_2}{R_1 + R_2} \right) \quad (3)$$

ADC Resolution: Quantizes an analog voltage into a digital integer. For an n -bit ADC with reference voltage V_{ref} , the step size (Resolution) is:

$$\text{Step Size (LSB)} = \frac{V_{ref}}{2^n} \quad (4)$$

The measured digital value D for an input voltage V_{in} is:

$$D = \text{round} \left(\frac{V_{in}}{\text{Step Size}} \right) = \text{round} \left(V_{in} \cdot \frac{2^n}{V_{ref}} \right) \quad (5)$$

2.2 Signals and Interface Elements

- **Digital vs. Analog:** Digital signals are binary (0 or 1, representing 0V and 3.3V/5V). Analog signals vary continuously (e.g., LDR resistance changing with light).
- **PWM (Pulse Width Modulation):** Mimics analog outputs from a digital pin by switching the pin ON and OFF very rapidly. The ratio of ON time to the period is called the **Duty Cycle** (D):

$$V_{eff} = D \cdot V_{max} \quad (6)$$

- **DHT11 Sensor:** A digital sensor measuring temperature and humidity, sending data over a single-wire digital protocol.
- **Relay:** An electromagnetic switch allowing a low-power microcontroller pin to switch high-voltage/high-current AC devices.

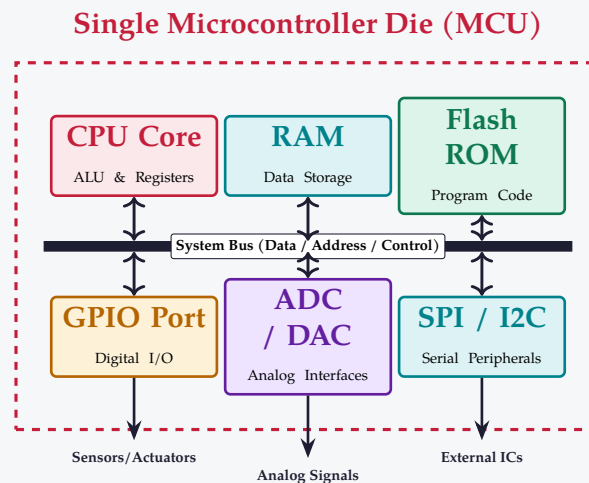
3. Prototyping Embedded Platforms

IoT designs select hardware platforms based on processing power, OS requirements, and connectivity.

3.1 Embedded Computing Basics

- **Microcontroller (MCU):** A single integrated circuit containing a CPU, RAM, Flash memory, and I/O peripherals (GPIO, ADC, SPI, I2C). Low-power, real-time execution. Example: ATmega328P.
- **Microprocessor (MPU):** Contains only the CPU. Requires external RAM, Flash, and support ICs. Runs complex Operating Systems (Linux). High power consumption. Example: ARM Cortex-A72.

Microcontroller Internal Architecture (System Bus Model)



3.2 Hardware Platforms Overview

1. Arduino (Microcontroller-Based):

- Powered by the **ATmega328P** 8-bit MCU operating at 16 MHz, containing 32 KB Flash, 2 KB SRAM, and 1 KB EEPROM.
- Programmed in C++ using bare-metal register manipulation (e.g. setting `DDRB |= (1 << 5)` to configure Pin 13 as output, and writing to `PORTB` to toggle pins) compiled via `avr-gcc`.
- Ideal for low-power sleep modes ($< 15 \mu\text{A}$ in power-down mode) and deterministic, timing-critical sensor polling.

2. Raspberry Pi (Single-Board Computer):

- Powered by a Broadcom **System-on-Chip (SoC)** containing a quad-core 64-bit ARM Cortex processor, up to 8 GB RAM, and GPU.
- Runs a fully pre-emptive Linux OS (Raspberry Pi OS) from an SD card. This introduces OS scheduling latency (non-deterministic) but allows full network stacks, file systems, and high-level runtimes (Python, Node.js).
- Used as local edge gateways, database hosts, and for CPU-intensive tasks like computer vision or edge AI inference.

3. BeagleBone Black (PRU Co-processors):

- Features a 1 GHz ARM Cortex-A8 processor running Linux, augmented with two auxiliary **PRUs (Programmable Real-Time Units)**.
- Each PRU is a 32-bit RISC core operating at 200 MHz with direct access to hardware I/O pins and shared RAM, operating independently of the Linux kernel's interrupts or process schedules.
- Solves the "non-deterministic OS" issue, enabling sub-microsecond GPIO toggling, precise high-speed industrial motor control, and custom hardware protocol decoding.

4. Electric Imp (Cloud-Integrated Module):

- A secure hardware module featuring an ARM Cortex processor and Wi-Fi interface.

- Utilizes a **Cloud-Device Split Model**: Code is split into a local *Device* script (interfacing with pins) and a cloud-based *Agent* script (running in Electric Imp's secure VM, handling HTTPS/OAuth).
- Programmed in the **Squirrel** scripting language. Features built-in secure boot, automated secure OTA firmware updates, and managed cloud routing.

5. Mobile Phone and Tablets (Sensors & Application Runtimes):

- High-performance mobile devices equipped with gigabytes of RAM, multi-core CPUs, and multi-Gbps cellular/Wi-Fi/Bluetooth networking.
- Host rich integrated sensors (dual-frequency GPS, 3-axis accelerometers, gyroscopes, magnetometers, ambient light sensors, cameras, and microphones).
- Used for rapid application-level prototyping, mobile dashboard display, and utilizing internal sensors as edge input devices.

6. Plug Computing (Always-On Mini-Servers):

- Compact, wall-plug-sized, always-on servers containing a low-power MPU (e.g. Marvell Kirkwood ARM), RAM, flash storage, and an Ethernet/Wi-Fi connection.
- Run headless Linux server distributions (e.g., Debian).
- Act as quiet, unobtrusive, low-power local gateways, print servers, and local home automation hub controllers.

Quick Revision — Module 3

Embedded Hardware Platforms | OE-EC803A

Key Formulas and Metrics at a Glance

Ohm's Law	$V = I \cdot R$ (Volts, Amperes, Ohms)
Power dissipation	$P = V \cdot I = I^2 \cdot R$ (Watts)
PWM Effective Voltage	$V_{\text{eff}} = D \cdot V_{\text{max}}$ ($D = \text{Duty Cycle} \in [0, 1]$)
ADC Resolution	$V_{\text{step}} = \frac{V_{\text{ref}}}{2^N - 1}$ (N -bit Analog-to-Digital Converter)

One-Line Definitions

- **Microcontroller:** An IC integrating CPU, memory, and peripheral interfaces on a single chip.
- **Single-Board Computer:** A complete computer built on a single circuit board, running an operating system.
- **PRU (BeagleBone):** Co-processors that run independently of the main OS to execute real-time I/O tasks.
- **PWM:** A technique to simulate analog voltages by pulsing a digital output at varying widths.
- **Relay:** An electromechanical switch used to control high-power circuits with low-power digital signals.

Frequently Asked / Viva Questions

Q1. What is the difference between a microcontroller and a microprocessor?

A microcontroller integrates the CPU, RAM, Flash memory, and I/O peripherals on a single silicon chip (System-on-Chip). A microprocessor contains only the CPU core and requires external components (RAM, Storage, Ethernet ICs) to function.

Q2. Why are microcontrollers preferred over microprocessors for simple IoT sensor nodes?

Microcontrollers consume far less power (milliwatts vs. watts), support deep sleep modes that allow battery operation for years, are much cheaper, and run code directly on bare-metal without the boot latency of an operating system.

Q3. What are the PRUs in a BeagleBone Black, and why are they useful?

The Programmable Real-Time Units (PRUs) are two auxiliary 32-bit RISC cores running at 200MHz independently of the main Linux OS. They allow BBB to

Frequently Asked / Viva Questions (contd.)

perform fast, deterministic real-time I/O tasks without scheduling delays caused by the Linux kernel.

Q4. Explain the concept of PWM and how it is used to dim an LED.

PWM (Pulse Width Modulation) switches a digital pin between high (5V) and low (0V) states rapidly. By changing the Duty Cycle (the fraction of time the pin is high), the effective average voltage varies continuously. This average voltage controls the LED brightness.

Q5. What is a Single-Board Computer (SBC)? Give an example.

A Single-Board Computer is a complete, functioning computer built on a single circuit board, incorporating a microprocessor, memory, networking, and USB ports. An example is the Raspberry Pi, which runs a Debian-based Linux OS.

Q6. How does a relay isolate a microcontroller from high-voltage appliances?

A relay uses an electromagnet. When the microcontroller energizes the internal coil (isolated low-voltage side), the magnetic field pulls a mechanical contact switch closed on the high-voltage side. There is no electrical connection between the two sides.

Q7. Why is community support considered an important parameter when choosing prototyping hardware?

IoT development moves fast. A large active community ensures open-source driver libraries are available for almost any sensor, troubleshooting forum threads exist for common bugs, and development boards are widely stocked.

Q8. What is Electric Imp, and how does it handle programming?

Electric Imp is a secure IoT module integrating Wi-Fi hardware with a managed cloud. It is programmed in the Squirrel scripting language, allowing developers to write both local device code and cloud agent code in a single framework.

Comparison Snapshot

Arduino vs. Raspberry Pi vs. BeagleBone Black:

Feature	Arduino Uno	Raspberry Pi 4	BeagleBone Black
Type	Microcontroller (MCU)	Single-Board Computer (SBC)	Industrial SBC
Processor	8-bit ATmega328P (16MHz)	64-bit Quad ARM (1.5GHz)	32-bit ARM (1GHz) + 2x PRUs
Memory	2 KB SRAM, 32 KB Flash	2GB / 4GB / 8GB LPDDR4	512MB DDR3, 4GB eMMC
Operating System	None (Bare-Metal)	Yes (Linux / Raspberry Pi OS)	Yes (Debian Linux)
Power Draw	Very Low (50mW)	Medium-High (3W to 5W)	Medium (1.5W to 2.5W)
Best Use Case	Analog sensor read, relays.	Computer vision, IoT gateway.	Real-time industrial automation.

Module 4

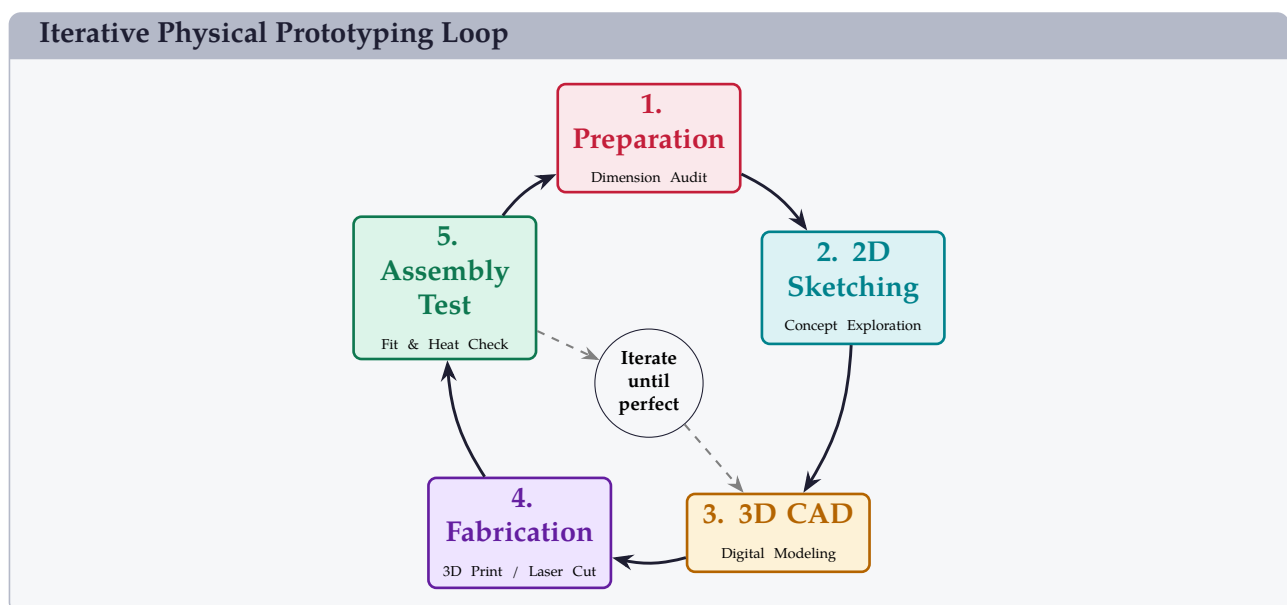
The Physical Prototyping Process

1. The Physical Prototyping Process

Physical design defines the mechanical structure, look, and user feel of an IoT device. An enclosure must protect the internal electronics while providing access to pins, ports, and sensors.

1.1 Prototyping Workflow Steps

- **Preparation:** Gathering circuit dimensions, mounting hole coordinates, and connectors.
- **Sketching & Exploration:** Creating rapid 2D hand sketches to study button placements and accessibility.
- **Iteration:** Generating digital CAD models and manufacturing initial prototypes.
- **Testing:** Assembling the electronics inside the case to check for connector alignment, heat dissipation, and physical tolerances.



2. Prototyping Methods

Various low-fidelity and high-fidelity fabrication methods are used depending on the design stage.

2.1 Non-Digital Prototyping (Low-Fidelity)

Before using digital machines, designers build fast, manual models using cardboard, foam core, acrylic sheets, and modeling clay.

- **Advantages:** Extremely low cost, zero machine wait time, and allows testing basic dimensions in a few minutes.
- **Drawback:** Low precision, fragile, and cannot be used for mechanical testing.

2.2 Laser Cutting (2D Vector Fabrication)

Laser cutting uses a high-power laser beam to cut or engrave sheet materials (acrylic, MDF, plywood, cardboard) based on 2D vector drawing profiles.

- **Vector Layouts:** Drawn in programs like Inkscape or AutoCAD using closed paths.
- **Kerf:** The width of the material that is vaporized by the laser beam during cutting (typically 0.1mm to 0.3mm). To create tight press-fit or finger joints, the design dimensions must compensate for this kerf allowance.
- **Joint Assembly Types:**
 - *Finger/Box Joints:* Interlocking rectangular teeth along edges, glued or press-fit.
 - *T-Slot Joints:* A slot receives a nut, and a perpendicular screw pulls the interlocking face tight. Excellent for structures needing frequent disassembly.
 - *Mortise and Tenon:* A tab (tenon) fits tightly into a matching slot (mortise) to align panels.

2.3 3D Printing (Additive Manufacturing)

3D Printing (Additive)

3D Printing is an additive manufacturing process that builds 3D solid structures layer-by-layer from a digital CAD file. The dominant technology is **FDM (Fused Deposition Modeling)**, which extrudes molten thermoplastic filament (PLA, ABS) through a heated nozzle.

Key 3D printing parameters that affect enclosure durability and print time:

- **Layer Height:** Controls vertical resolution (typically 0.1mm to 0.3mm). Smaller values give smoother surfaces but increase print time.
- **Infill Density & Patterns:** The internal grid structure, ranging from 0% (hollow) to 100% (solid). Enclosures typically use 15% to 30% infill. Patterns include *Honeycomb* (strongest in multiple axes), *Gyroid* (isometric strength and flexibility), and *Grid* (fastest printing).
- **Wall/Shell Thickness:** The thickness of the outer boundaries, defining enclosure strength and screw-mounting thread durability.

- **Materials:**

- *PLA (Polylactic Acid)*: Biodegradable starch-based polymer. Easy to print, low warping, but low temperature resistance (warps above 60°C).
- *ABS (Acrylonitrile Butadiene Styrene)*: Petroleum-based. Highly impact-resistant, high heat deflection, but requires a heated enclosure and emits toxic fumes during printing.
- *PETG (Polyethylene Terephthalate Glycol)*: Blends ease of PLA with durability of ABS. Excellent chemical and weather resistance, ideal for outdoor IoT nodes.

2.4 CNC Milling (Subtractive Manufacturing)

CNC (Computer Numerical Control) milling uses rotating cutting bits to carve away material from a solid block of plastic, wood, or metal (aluminum).

- **Subtractive Concept:** The opposite of 3D printing. It starts with a solid stock block and subtracts material using high-speed rotary cutters.
- **Milling Toolpaths:** G-code instructions define specific cutter movements:
 - *Pocketing*: Carving out internal hollow cavities inside a solid block (e.g. creating the box cavity for electronics).
 - *Contouring (Profiling)*: Cutting along outer boundary lines to separate the final piece from raw stock.
 - *Facing*: Machining flat surfaces on a block to ensure absolute thickness tolerance.
- **Applications:** Best for high-strength enclosures, metal heat-sink cases, and prototyping circuit boards (PCB isolation routing).

2.5 Repurposing and Recycling

Hacking off-the-shelf plastic junction boxes, containers, or toys by drilling holes for cables and sensors.

- **Advantages:** Provides immediate structural protection, is highly cost-effective, and reduces e-waste footprint during research stages.

Quick Revision — Module 4

Physical Design & Prototyping | OE-EC803A

One-Line Definitions

- **Kerf:** The thickness of the cut line or material slot vaporized by a laser cutter.
- **FDM:** A 3D printing process that builds objects by extruding layers of melted plastic filament.
- **Subtractive Manufacturing:** A fabrication process where material is carved away from a solid block.
- **Additive Manufacturing:** A fabrication process where material is joined layer-by-layer to build a structure.
- **Infill:** The internal supporting pattern printed inside a 3D object to save time and material.

Understanding the kerf is critical for press-fit joints where interlocking teeth must hold together without glue.

Frequently Asked / Viva Questions

Q1. What is the difference between additive and subtractive manufacturing?

Additive manufacturing (e.g. 3D printing) builds parts layer-by-layer from nothing, producing zero waste and allowing complex internal cavities. Subtractive manufacturing (e.g. CNC milling) carves parts out of a solid block of raw material, producing waste chips but yielding high strength and support for metals.

Q2. Explain the significance of “kerf” in laser cutting.

The kerf is the width of the cut slot created by the laser beam vaporizing material. If a laser has a kerf of 0.2mm, cutting a 10.0mm square will result in a piece that is 9.8mm wide. Designers must compensate for this kerf to create tight, interlocking press-fit joints.

Q3. Why is infill density in 3D printing rarely set to 100%?

A 100% infill uses massive amounts of filament and takes a very long time to print. Structural strength gains level off significantly beyond 40% infill. Enclosures are usually printed with 15% to 30% infill using internal patterns (like grid or gyroid) to maximize strength-to-weight ratio.

Q4. What is Fused Deposition Modeling (FDM)?

FDM is the most common 3D printing technology. It melts a plastic filament (like PLA or ABS) and extrudes it through a small nozzle, moving in X, Y, and Z axes to lay down thin layers that fuse together as they cool.

Frequently Asked / Viva Questions (contd.)

Q5. Why would a designer choose CNC milling over 3D printing for an industrial IoT gateway enclosure?

CNC milling allows using materials like structural aluminum. Aluminum enclosures provide superior mechanical protection, complete electromagnetic shielding (Faraday cage), and act as built-in heat sinks for high-power processors.

Q6. What is the purpose of low-fidelity physical prototyping using cardboard?

Low-fidelity prototyping allows designers to physically hold and check the dimensions of a device layout within minutes, helping catch major interface errors (like port placement blockage or button hand-reach issues) before executing expensive machine runs.

Q7. What is repurposing/recycling in physical design?

Repurposing refers to taking an existing mass-produced plastic box (like an electrical junction box or soap container) and drilling custom ports to house the IoT circuit. This saves cost, is weather-sealed, and reduces development time.

Q8. What is SLA (Stereolithography) 3D printing, and how does it differ from FDM?

SLA uses a liquid photopolymer resin that is cured layer-by-layer using a UV laser beam. It yields extremely high resolutions and smooth surface finishes compared to FDM, but the materials are more fragile and require post-print chemical cleaning.

Comparison Snapshot

3D Printing vs. CNC Milling vs. Laser Cutting:

Feature	3D Printing (FDM)	CNC Milling	Laser Cutting
Process	Additive (layer-by-layer).	Subtractive (carving).	2D Cutting / Engraving.
Dimensions	Complex 3D geometries.	3D shapes with tool limitations.	Flat 2D sheets (can fold/slot).
Materials	PLA, ABS, PETG.	Metals (Al, Steel), Wood, Acetal.	Acrylic, MDF, Plywood, Cardboard.
Wastage	Very Low (only support material).	High (residual chips and dust).	Low-Medium (sheet offcuts).
Setup Speed	Very Fast (load STL to printer).	Slow (requires CAM setups, tools).	Fast (load vector file).

Module 5

Prototyping Online Components (APIs)

1. Prototyping Online Components (APIs)

To connect physical hardware to cloud services, devices communicate with web application interfaces called APIs (Application Programming Interfaces).

1.1 RESTful APIs

REST API

A **REST (Representational State Transfer) API** is an architectural style for network applications that uses standard HTTP requests to manage resources. Each resource is identified by a URL (e.g. `'/devices/lamp1'`) and manipulated using standard HTTP verbs.

The primary HTTP verbs used in IoT APIs include:

- **GET:** Retrieve the status/value of a sensor or actuator.
- **POST:** Create a new status log or register a new device.
- **PUT / PATCH:** Update configuration properties or toggle state.
- **DELETE:** Remove a device instance from the registry database.

Common HTTP response status codes returned by IoT APIs:

- **200 OK:** Request succeeded (e.g. GET returned sensor reading).
- **201 Created:** Resource successfully created (e.g. POST registered a new device).
- **400 Bad Request:** Invalid syntax or malformed JSON payload.
- **401 Unauthorized:** Missing or invalid API key / authentication token.
- **404 Not Found:** Target device or resource URL does not exist.
- **500 Internal Server Error:** Server-side runtime database/connection failure.

Data is commonly formatted in **JSON (JavaScript Object Notation)** due to its low parsing overhead and human-readable, key-value structure:

```
{  
  "device_id": "sensor_node_04",  
}
```

```
"temperature": 26.5,  
"status": "active"  
}
```

1.2 Writing a New API (Backend Endpoint)

Writing a new REST API endpoint involves setting up a server runtime (e.g. Node.js with Express or Python with Flask) that accepts incoming JSON payloads, processes them, and returns standard HTTP response codes.

Express.js IoT Server-Side Logging API

Below is a standard server-side Node.js script that listens for HTTP `POST` requests from IoT sensor nodes and saves the reading:

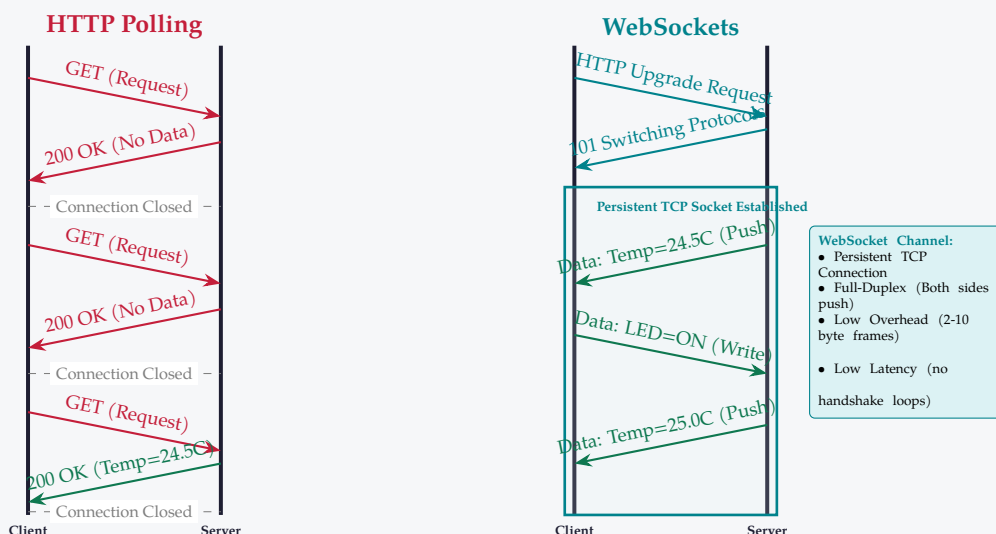
```
const express = require('express');  
const app = express();  
app.use(express.json()); // Parse JSON payloads  
  
app.post('/api/telemetry', (req, res) => {  
  const { device_id, temperature } = req.body;  
  
  if (!device_id || temperature === undefined) {  
    return res.status(400).json({ error: "Missing parameters" });  
  }  
  
  console.log(`Log from ${device_id}: ${temperature} C`);  
  // (In production: Insert data into MongoDB or PostgreSQL database)  
  
  res.status(201).json({  
    message: "Data logged successfully",  
    timestamp: new Date()  
  });  
});  
  
app.listen(3000, () => console.log('IoT Server running on port 3000'));
```

1.3 Real-Time Reactions: Polling vs. WebSockets

IoT systems need to push real-time events (e.g., triggering an alarm).

- **HTTP Polling:** The client periodically sends GET requests to the server (e.g., every 5 seconds) to check for updates. This generates significant network overhead and latency.
- **HTTP Long Polling:** The server holds the client's request open until new data is available, responding immediately, then closing the connection.
- **WebSockets:** Establishes a persistent, bi-directional TCP connection. Both client and server can send data packets at any time with minimal header overhead, enabling sub-millisecond real-time reactions.

HTTP Polling vs. WebSockets Communication Flow



1.4 Other Messaging Protocols

- **XMPP (Extensible Messaging and Presence Protocol):** XML-based instant messaging protocol. Heavy overhead but features strong built-in security and routing.
- **AMQP (Advanced Message Queuing Protocol):** An open standard queuing protocol designed for enterprise transaction messaging, providing high reliability.

2. Techniques for Writing Embedded Code

Microcontrollers are resource-constrained devices, requiring specialized programming practices.

2.1 Memory Management & Heap Fragmentation

Heap Fragmentation

Heap Fragmentation occurs when dynamic memory allocation (`'malloc()'` and `'free()'`) allocates and deallocates varying memory chunks over time, leaving small, unusable gaps in the heap. In microcontrollers with only a few kilobytes of RAM, this can eventually prevent large allocations, leading to program crashes.

To prevent fragmentation, embedded code should:

- Avoid dynamic memory (`'new'`, `'malloc'`, `'String'` objects).
- Prefer static global allocation (defining buffer arrays with fixed sizes at compile time) or stack-based local variables.

2.2 Power Management and Battery Life Optimization

Many IoT devices run on batteries, requiring strict power management:

- **Active Mode:** CPU running at full clock speed, radio transmitting. Draws high current

(10mA to 150mA).

- **Light Sleep:** CPU clock is gated/halted, RAM is preserved. Draws microamps ($100\mu\text{A}$ to $500\mu\text{A}$).
- **Deep Sleep:** CPU, RAM, and internal oscillators are completely shut down. Only an ultra-low-power Real-Time Clock (RTC) or external interrupt remains active. Draws nanoamps ($1\mu\text{A}$ to $15\mu\text{A}$). The device wakes up, reads sensors, transmits, and immediately sleeps again.

Duty Cycling & Battery Life

Duty Cycling is the ratio of active time (t_{active}) to the total period ($T = t_{\text{active}} + t_{\text{sleep}}$):

$$D = \frac{t_{\text{active}}}{t_{\text{active}} + t_{\text{sleep}}} \quad (7)$$

The average current draw (I_{avg}) is calculated as:

$$I_{\text{avg}} = D \cdot I_{\text{active}} + (1 - D) \cdot I_{\text{sleep}} \quad (8)$$

Battery lifetime (L in hours) for a battery with capacity C (in mAh) is:

$$L = \frac{C}{I_{\text{avg}}} \quad (9)$$

Numerical Example: IoT Battery Life Calculation

Scenario: An IoT sensor node wakes up once every 10 minutes (600 seconds) for a duration of 2 seconds to read a sensor and transmit it via Wi-Fi.

- Active Current (I_{active}) = 80 mA
- Sleep Current (I_{sleep}) = $15\mu\text{A} = 0.015\text{ mA}$
- Battery Capacity (C) = 2400 mAh (e.g. standard AA cells)

Step 1: Calculate Duty Cycle (D)

$$D = \frac{2\text{ s}}{600\text{ s}} = 0.00333 \quad (0.33\%) \quad (10)$$

Step 2: Calculate Average Current (I_{avg})

$$I_{\text{avg}} = (0.00333 \cdot 80\text{ mA}) + ((1 - 0.00333) \cdot 0.015\text{ mA}) \quad (11)$$

$$I_{\text{avg}} \approx 0.2667\text{ mA} + 0.0150\text{ mA} = 0.2817\text{ mA} \quad (12)$$

Step 3: Calculate Battery Lifetime (L)

$$L = \frac{2400\text{ mAh}}{0.2817\text{ mA}} \approx 8519.7\text{ hours} \approx 355\text{ days} \quad (13)$$

Thus, by employing aggressive duty cycling, a standard battery can sustain the node for approximately one full year despite its high 80mA active transmission current.

2.3 Embedded Debugging Techniques

Debugging on microcontrollers is challenging because they lack screens.

- **Serial Debugging:** Printing variable values to a host computer serial monitor (`'Serial.println()'`). Simple but introduces delay (timing bugs).
- **In-Circuit Emulation & JTAG:** A hardware interface (Joint Test Action Group) that connects directly to the CPU core, allowing developers to set hardware breakpoints, inspect memory registers, and step through code line-by-line.

Quick Revision — Module 5

Online Components & Embedded Programming | OE-EC803A

Key Formulas and Metrics at a Glance

Duty Cycle	$D = \frac{t_{\text{active}}}{t_{\text{active}} + t_{\text{sleep}}}$ (ratio of active execution time)
Average Current	$I_{\text{avg}} = D \cdot I_{\text{active}} + (1 - D) \cdot I_{\text{sleep}}$ (power profile)
Battery Lifetime	$L = \frac{\text{Battery Capacity (mAh)}}{I_{\text{avg}} \text{ (mA)}}$ (operation hours)

One-Line Definitions

- **REST:** A stateless web architectural style that relies on standard HTTP methods (GET/POST).
- **WebSocket:** A persistent, full-duplex TCP socket connection for real-time web communication.
- **Duty Cycling:** The practice of powering down the device CPU and radio for periodic sleep intervals.
- **Deep Sleep:** The lowest power state of an MCU, turning off all internal blocks except the RTC wakeup timer.
- **JTAG:** A hardware port standard used for direct in-circuit programming, testing, and debugging.

Frequently Asked / Viva Questions

Q1. Why is dynamic memory allocation ('malloc') highly discouraged in microcontroller code?

Microcontrollers have extremely limited RAM (often a few kilobytes) and no virtual memory management. Repeated allocations and deallocations leave small gaps of unused memory on the heap (heap fragmentation). Eventually, a request for a contiguous memory block fails, causing a system crash.

Q2. Compare HTTP Polling and WebSockets for real-time sensor updates.

HTTP Polling requires the client to repeatedly send new request handshakes and headers, creating heavy network congestion and latency. WebSockets establishes a single persistent TCP connection, enabling bi-directional, real-time message streams with minimal packet overhead.

Q3. Calculate the average current and battery life of an IoT node with a 1000mAh battery. It draws 100mA for 2 seconds when active, and 10μA (0.01mA) for 58 seconds during sleep.

Frequently Asked / Viva Questions (contd.)

(1) Total Period $T = 2 + 58 = 60$ seconds. (2) Duty Cycle $D = 2/60 = 0.0333$ (3.33%). (3) Average Current $I_{\text{avg}} = (0.0333 \times 100) + (0.9667 \times 0.01) = 3.33 + 0.0097 \approx 3.34\text{mA}$. (4) Lifetime $L = 1000/3.34 \approx 299.4$ hours (≈ 12.5 days).

Q4. What occurs during deep sleep in an ESP8266 or ESP32 microcontroller?

During deep sleep, the CPU, RAM, and internal Wi-Fi radio are completely powered off. The only part of the chip that remains powered is the low-power RTC timer and select GPIO wake pins. The CPU must re-initialize (reboot) from scratch when waking up.

Q5. What is a REST API HTTP POST request used for in IoT?

A POST request is used to push data from the client to the server, creating a new database record (e.g., posting a new temperature reading log at '/api/v1/telemetry').

Q6. Explain the JTAG debug interface.

JTAG is a hardware debugging protocol. It uses dedicated physical pins on the microcontroller to interface directly with the internal processor tap, allowing developers to halt CPU execution, step through commands, and read register states.

Q7. What is JSON, and why is it preferred over XML in IoT applications?

JSON is a lightweight key-value text format for representing structured data. It has a much smaller character count than XML (no closing tags), which saves bandwidth, and requires significantly less CPU memory and time to parse.

Q8. How does duty cycling extend device battery life?

Sensors rarely need to transmit data continuously. By sleeping for 99% of the time and only waking up for a few seconds to sample and transmit data, the average current draw is reduced to microamps, extending battery life from days to years.

Comparison Snapshot**HTTP Polling vs. WebSockets:**

Feature	HTTP Polling	WebSockets
Connection Type	Temporary; opened and closed per request.	Persistent; remains open until closed.
Communication	Half-duplex (client pull only).	Full-duplex (bi-directional push/pull).
Network Overhead	High (repeats TCP handshakes and HTTP headers).	Minimal (small frame header of 2-10 bytes).
Real-Time Latency	High (depends on polling frequency).	Extremely Low (sub-millisecond).
Best Use Case	Static data dashboards.	Real-time alerts, remote live control.

Module 6

IoT Business Models & Startups

1. IoT Business Models & Startups

Transitioning an IoT prototype into a commercial product requires establishing sustainable business value.

1.1 A Short History of Business Models

Historically, business models revolved around the **transactional model** (Value-Transaction-Exchange), where companies designed, manufactured, and sold physical products as a one-time purchase. In the digital age, this evolved into the **subscription/service-based model** (e.g. SaaS). IoT merges these two histories: it combines the transactional sale of physical hardware with the ongoing subscription-based service model (Product-as-a-Service), creating a continuous customer relationship.

1.2 Who is the Business Model For?

An IoT business model serves three main target audiences:

- **Investors:** Demonstrates the viability, return on investment (ROI), scalability, and path to profitability (amortizing high upfront tooling and NRE costs).
- **Partners and Suppliers:** Clarifies roles in the value chain, such as PCB manufacturing houses, cloud providers, and distribution networks.
- **Internal Team:** Aligns the engineering, sales, and support departments on customer needs, product value propositions, and resource allocations.

1.3 Business Model Canvas in IoT

The **Business Model Canvas** defines a startup's value chain across 9 building blocks. IoT products alter these blocks:

1. **Value Proposition:** Continuous software updates, predictive maintenance, and remote automation rather than just static physical utility.
2. **Revenue Streams:** Low upfront hardware margins compensated by recurring subscription models (SaaS / Hardware-as-a-Service).
3. **Customer Segments:** Distinct B2C (consumers desiring convenience) vs. B2B (enterprises desiring efficiency/analytics).

4. **Customer Relationships:** Continuous digital engagement via apps and automated support, rather than one-time transaction endpoints.
5. **Channels:** Digital app stores and direct e-commerce, paired with physical retail and distribution channels.
6. **Key Activities:** Firmware maintenance, OTA software updates, security patch deployment, and cloud server scaling.
7. **Key Resources:** Proprietary sensor algorithms, cloud database infrastructure, engineering talent, and manufacturing supply chains.
8. **Key Partners:** Cloud platform providers (AWS, Azure), silicon chip vendors, contract manufacturers, and network carrier operators.
9. **Cost Structure:** High initial NRE (Non-Recurring Engineering) tooling costs, bill of materials (BOM), cloud bandwidth, and continuous server hosting costs.

1.4 Lean Startup Methodology

- **MVP (Minimum Viable Product):** Building a prototype containing only the core feature that solves the user's primary problem.
- **Feedback Loop:** The iterative **Build-Measure-Learn** loop helps refine the MVP based on real user feedback before funding expensive manufacturing tooling.

1.5 Funding Options

IoT startups fund early stages via:

- **Crowdfunding (Kickstarter):** Users pre-order the product, verifying market demand and funding initial production runs.
- **Venture Capital (VC) & Bootstrapping:** Institutional funding for scaling versus self-funding from revenues.

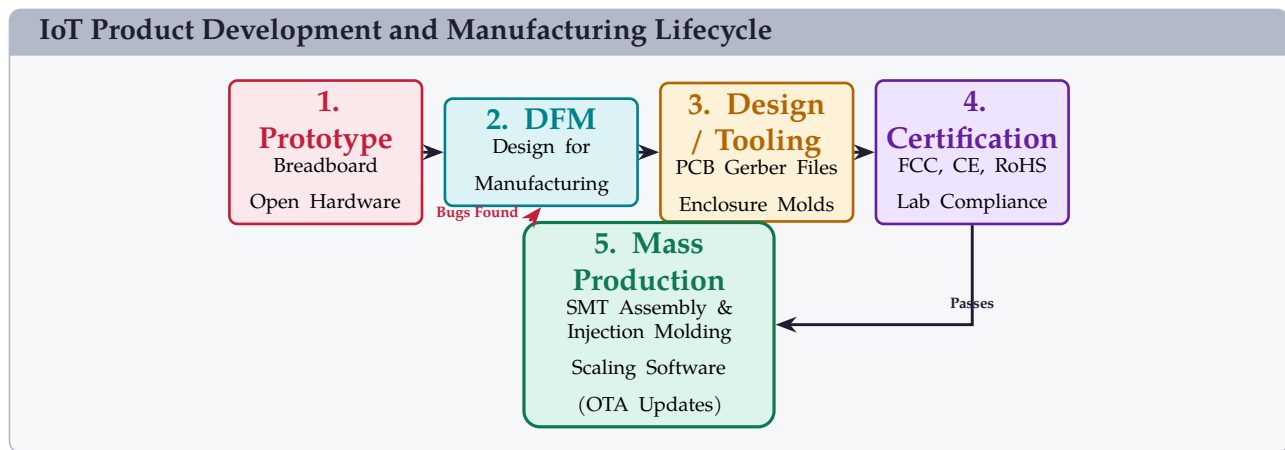
2. Moving to Manufacture

Mass-producing electronic hardware involves transitioning from breadboards to industrial assembly lines.

2.1 What are you Producing? & Designing Kits

Before manufacturing, developers must define the physical scope of their product:

- **What are you Producing?:** Is the product a raw sub-assembly component, an industrial sensor node, or a polished end-user consumer product? This determines the enclosure design and target user experience.
- **Designing Kits:** Many IoT systems begin as developer kits (e.g. SparkFun/Adafruit starter kits). Designing a kit requires organizing components, color-coding wire connectors, providing clear assembly diagrams, and bundling software library links to reduce the user's first-time configuration friction.



2.2 Designing and Manufacturing PCBs

1. **Schematic Capture:** Drawing the logical connectivity diagram between components using CAD software (e.g. KiCad, Altium).
2. **PCB Layout Routing:** Arranging physical component footprints and routing copper traces to connect pins. Features include:
 - *Trace Width:* Thicker traces are routed for power lines to handle current without overheating; thin traces are used for signals.
 - *Vias:* Plated copper cylinders that bridge traces between different layer planes.
 - *Ground Planes:* Large copper planes added to shield signals from EMI and provide a stable reference voltage.
3. **PCB Fabrication Steps:**
 - *Etching:* Removing unwanted copper from substrate using chemical etchants (e.g. Ferric Chloride or Ammonium Persulfate).
 - *Drilling & Electroplating:* Drilling mounting and via holes, then electroplating them with copper to establish layer connections.
 - *Solder Mask:* Applying a polymer layer (usually green) to insulate copper traces and prevent solder bridging.
 - *Surface Finish:* Coating exposed copper pads to prevent oxidation (e.g. **HASL** - Hot Air Solder Leveling, cheap; or **ENIG** - Electroless Nickel Immersion Gold, high-quality flat pads for fine SMT components).
4. **Assembly (SMT vs. Through-Hole):**
 - **SMT (Surface Mount Technology):** Components (SMD) are placed on solder paste and reflowed in an oven. Automated via pick-and-place machines; allows dual-sided, high-density designs.
 - **Through-Hole Technology (THT):** Component pins are inserted through holes and wave-soldered. Essential for connectors and heavy power modules due to high mechanical strength.

2.3 Mass-Producing Enclosures (Injection Molding)

Injection Molding

Injection Molding is the primary manufacturing process for plastic enclosures. Molten plastic (ABS, Polycarbonate) is injected under high pressure into a custom-machined steel or aluminum mold cavity, where it cools and solidifies into the final case.

Key design constraints and manufacturing parameters for injection-molded parts:

- **High Tooling Costs:** Machining the steel molds (tooling) is extremely expensive (often \$10,000 to \$100,000 NRE costs), requiring massive production volumes to amortize the setup cost.
- **Draft Angles:** Vertical walls must be tapered slightly (typically 1° to 2°) to allow the solidified plastic part to eject easily from the steel mold without scratching.
- **Common Molding Defects:**
 - *Sink Marks:* Shallow depressions formed on thick wall sections due to slower cooling and volumetric shrinkage. Fixed by maintaining uniform wall thickness.
 - *Warping:* Distortion of flat surfaces caused by non-uniform cooling rates across different sections of the part.
 - *Knit Lines (Weld Lines):* Lines where multiple molten plastic flow fronts meet and fail to fuse completely, representing structural weak spots.
 - *Flash:* Excess plastic leakage that escapes along the parting line of the mold halves due to low clamping pressure.

2.4 Hardware Certification

Before selling electronic products, they must satisfy regulatory standards:

- **FCC (Federal Communications Commission) Certification:** Required for products sold in the United States. It regulates electromagnetic compatibility (EMC):
 - *Intentional Radiators:* Devices that actively broadcast RF signals (e.g. Wi-Fi, Bluetooth, cellular). Require full, expensive transmitter testing and registration.
 - *Unintentional Radiators:* Devices that do not intentionally broadcast radio signals but generate internal high-frequency signals (> 9 kHz) like CPU clocks. Require self-declaration or verification.
- **CE (Conformité Européenne) Marking:** Mandatory for products in the European Economic Area. Declares that the product complies with health, safety, and environmental protection directives (such as the Radio Equipment Directive - RED, and EMC Directive).
- **RoHS (Restriction of Hazardous Substances):** Restricts the use of specific hazardous substances in electrical and electronic equipment (specifically Lead, Mercury, Cadmium, Hexavalent Chromium, and flame retardants like PBB/PBDE). Requires lead-free solder and components.

2.5 Manufacturing Cost Concepts

Managing economics is critical when moving to mass manufacture:

- **BOM (Bill of Materials):** A complete, hierarchical spreadsheet list of every raw component, IC, circuit board, enclosure part, connector, screw, and label required to manufacture one single unit, including supplier part numbers and cost-breaks.
- **NRE (Non-Recurring Engineering):** The one-time, upfront development cost of design, PCB routing labor, test fixture setup, compliance testing, and steel injection molds.

Amortized Unit Cost Formula

The total per-unit production cost (U) for a manufacturing run of N units is:

$$U = \text{BOM} + \frac{\text{NRE}}{N} \quad (14)$$

As $N \rightarrow \infty$, the amortized cost approaches the BOM cost. Small runs (low N) have high per-unit costs due to the unamortized NRE.

2.6 Software Scaling and OTA Updates

When scaling from 1 to 10,000 devices:

- Devices must support **OTA (Over-the-Air) Firmware Updates** to patch security bugs remotely over Wi-Fi/cellular without physical access.

3. Ethics, Security & Environmental Solutions

Connecting billions of physical devices introduces deep social and environmental challenges.

3.1 Characterizing the Internet of Things (Ethical Perspective)

The ethical challenges of IoT are characterized by three core traits:

- **Pervasiveness (Ubiquity):** Devices are embedded in private spaces (homes, offices, bodies) and gather data continuously and invisibly, leaving users unaware of the data collection.
- **Physical Agency:** Unlike the traditional internet (which only displays information), IoT has physical actuators that control real-world environments. A breach can cause immediate physical danger (e.g., unlocking a smart door or disabling a smart brake).
- **Data Ownership and Asymmetry:** A massive imbalance exists between the user (who generates the data) and the manufacturer/service provider (who aggregates, analyzes, and often sells the data).

3.2 Privacy and Control

IoT devices gather intimate details (voices, camera feeds, occupancy).

- **Security:** Many IoT devices have weak security (default passwords, unencrypted firmware). Attackers can hijack them to form **Botnets** (e.g. Mirai botnet) to launch DDoS attacks on servers.
- **Control:** Who owns the data collected by a smart thermostat? Device makers must ensure localized user consent and encrypt data during transmission (using SSL/TLS or DTLS).

3.3 Environmental Impact (E-Waste)

- Millions of short-lived IoT tags and smart devices end up in landfills, leading to heavy metal (lead, mercury) contamination in soil.
- **Solutions:** Designing for repairability, using modular component slots, and using biodegradable paper substrates or organic components for temporary IoT sensors.

Quick Revision — Module 6

Prototype to Reality | OE-EC803A

Key Cost and Amortization Formulas

Total Unit Cost	$\text{Cost}_{\text{unit}} = \text{BOM Cost} + \text{Assembly Cost} + \frac{\text{NRE Cost}}{N}$
BOM Cost	$\text{BOM} = \sum (\text{Component Cost}_i \times \text{Quantity}_i) \quad (\text{list of materials})$
NRE Costs	Amortized over production run volume N . High volume shrinks NRE impact.

One-Line Definitions

- **Gerber File:** Standard file format containing physical layer instructions for manufacturing PCBs.
- **SMT:** Automated technology where electronic components are mounted directly onto the surface of a PCB.
- **Draft Angle:** A slight taper (1-2 degrees) on vertical walls of a molded case to allow easy ejection.
- **RoHS:** A directive restricting hazardous lead, mercury, and cadmium in electronic components.
- **OTA Update:** The process of upgrading a device's firmware wirelessly over the network.

Frequently Asked / Viva Questions

Q1. What is a Gerber file, and why is it important in PCB manufacturing?

A Gerber file is a set of open 2D vector images showing the copper traces, solder masks, silkscreen labels, and drill paths for each layer of the PCB. PCB fabrication houses require Gerber files to program their automated drilling and etching machines.

Q2. What is the difference between SMT and Through-Hole assembly?

SMT (Surface Mount) components are soldered directly to pad surfaces on the board, allowing automated high-density assembly. Through-Hole components have long wire leads inserted through holes in the board and soldered on the opposite side, which provides superior mechanical strength for heavy components like terminals and power relays.

Q3. Why are upfront tooling costs for plastic injection molding extremely high?

The molds must be custom-machined out of solid blocks of hardened tool steel

Frequently Asked / Viva Questions (contd.)

using high-precision CNC machines to withstand thousands of high-pressure, high-temperature plastic injection cycles. This setup cost (NRE) is high but reduces the cost per part to cents at high volumes.

Q4. What is a draft angle in injection molding, and why is it required?

A draft angle is a slight taper (typically 1° to 2°) added to the vertical walls of the plastic enclosure design. Without this taper, the cooling plastic shrinks slightly around the mold cavities, causing high friction and damage during ejection.

Q5. Explain the role of FCC certification for IoT devices.

The FCC (Federal Communications Commission) regulates RF emissions. Any digital device switching signals at high speeds or using wireless transmitters must undergo testing to prove it does not emit electromagnetic interference (EMI) that could disrupt other communications.

Q6. What is a Bill of Materials (BOM) in hardware manufacturing?

A BOM is a structured table listing every single component, PCB substrate, connector, case, fastener, label, and packaging box required to manufacture a single product, along with manufacturer part numbers and costs.

Q7. How can IoT devices pose a security threat to other internet services?

Many consumer IoT devices lack secure firewalls and default login credentials. Attackers can scan the web and infect thousands of these vulnerable devices with malware, forming a botnet (such as Mirai) to launch massive Distributed Denial of Service (DDoS) attacks.

Q8. What is the Product-as-a-Service (PaaS) business model in IoT?

In PaaS, rather than paying a large one-time fee to own hardware, customers buy hardware cheaply and subscribe to a service (recurring revenue stream). The value is delivered through ongoing cloud monitoring, alerts, and automatic software updates.

Comparison Snapshot**3D Printing vs. Injection Molding for Enclosure Production:**

Feature	3D Printing (Prototype)	Injection Molding (Mass Production)
Upfront Tooling Cost	Zero (direct print from STL).	Extremely High (\$10,000+ for steel molds).
Unit Cost	Medium-High (PLA filament cost per hour).	Extremely Low (cents per plastic part).
Production Cycle Time	Slow (several hours per enclosure).	Very Fast (seconds per enclosure injection).
Design Constraints	Layer lines, support structures needed.	Draft angles, uniform wall thickness required.
Best Volume	1 to 50 units (prototypes).	5,000+ units (mass market).